



# Security Audit

Perena (DeFi)

# Table of Contents

|                                   |    |
|-----------------------------------|----|
| Executive Summary                 | 4  |
| Project Context                   | 4  |
| Audit Scope                       | 7  |
| Security Rating                   | 8  |
| Intended Smart Contract Functions | 9  |
| Code Quality                      | 12 |
| Audit Resources                   | 12 |
| Dependencies                      | 12 |
| Severity Definitions              | 13 |
| Status Definitions                | 14 |
| Audit Findings                    | 15 |
| Centralisation                    | 40 |
| Conclusion                        | 41 |
| Our Methodology                   | 42 |
| Disclaimers                       | 44 |
| About Hashlock                    | 45 |

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

## Executive Summary

The Perena team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

## Project Context

Perena is a blockchain / DeFi project offering USD\*, a programmable, utility-driven stablecoin engineered to capture value from protocol activity, integrate natively with liquidity pools and yield strategies, and foster a community-driven value loop. The platform aggregates stablecoin liquidity across protocols into a unified layer to address fragmentation in the stablecoin market, backs USD\* with a diversified allocation to DeFi and RWA to enhance liquidity, usability, and rewards.

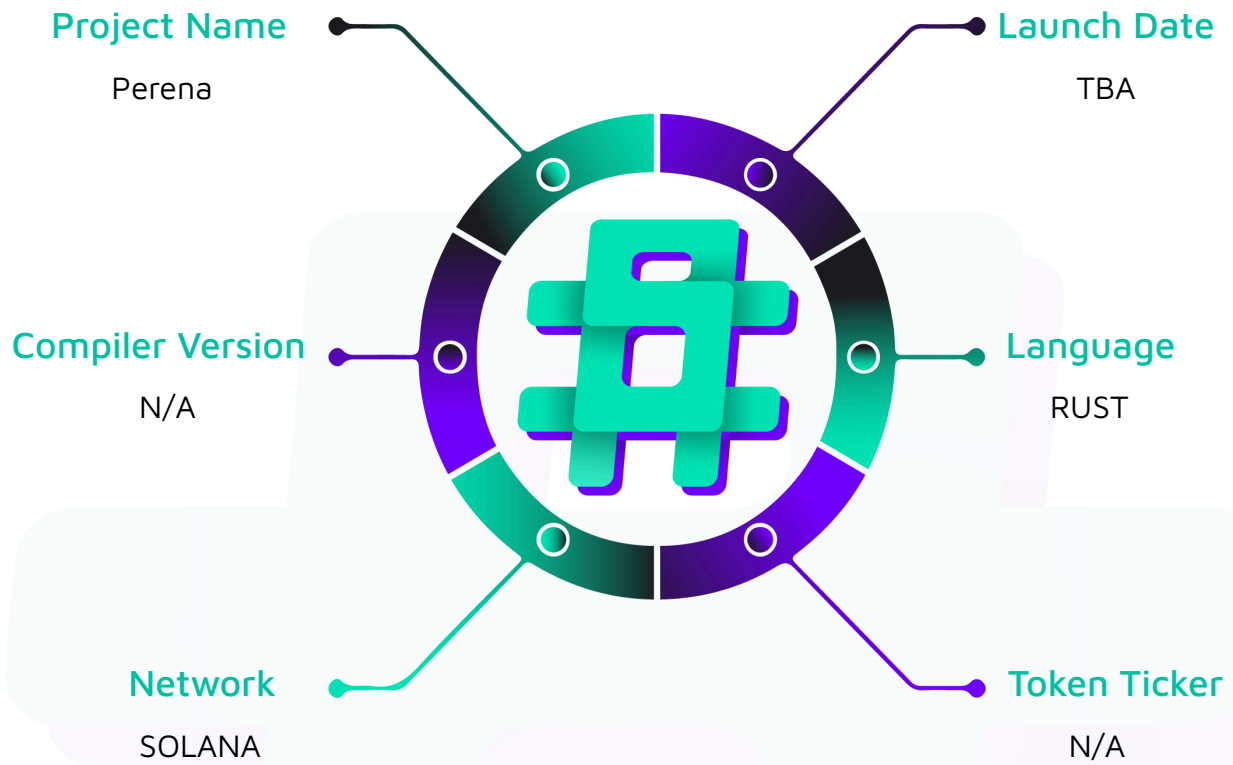
**Project Name:** Perena

**Project Type:** Defi

**Website:** <https://perena.org/>

**Logo:**



**Visualised Context:**

## Project Visuals:



## Audit Scope

We at Hashlock audited the Rust code within the Perena project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

| Description                   | Perena Smart Contracts                   |
|-------------------------------|------------------------------------------|
| Platform                      | Solana / Rust                            |
| Audit Date                    | October, 2025                            |
| Program 1                     | programs/bankineco                       |
| Audited GitHub Commit Hash    | 97b69ae759d3cc0cced851bf82e604d06e991b54 |
| Fix Review GitHub Commit Hash | 59d95783b4f345f55d2281414f5ccdae9f5476ee |

# Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

## Hashlock found:

3 High severity vulnerabilities

6 Medium severity vulnerabilities

8 Low severity vulnerabilities

**Caution:** Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

## Intended Smart Contract Functions

| Claimed Behaviour                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Actual Behaviour                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| <p><b>instructions/admin/</b></p> <ul style="list-style-type: none"> <li>- <b>create_bank.rs</b> - Initialize BankState PDA with vault registry, accounting fields and mint reference</li> <li>- <b>create_gen_vault.rs</b> - Create VaultGenState PDA, initialize vault ATA, register in bank's vault array</li> <li>- <b>create_oracle_state.rs</b> - Setup OracleGenState with two oracles, initial price and consensus parameters</li> <li>- <b>create_team_state.rs</b> - Create TeamGenState PDA and fee collection ATA for vault management</li> <li>- <b>name_bank_manager.rs</b> - Update bank manager authority</li> <li>- <b>name_bank_risk_manager.rs</b> - Update bank risk manager authority</li> <li>- <b>name_vault_risk_manager.rs</b> - Update vault risk manager authority</li> <li>- <b>name_external_yield_manager.rs</b> - Set external yield manager for UnCollateralizedExternalYield vaults</li> <li>- <b>bank_circuit_breaker.rs</b> - Halt and resume bank operations</li> <li>- <b>vault_circuit_breaker.rs</b> - Halt and resume vault operations</li> <li>- <b>change_fees.rs</b> - Update vault fee parameters</li> <li>- <b>update_price.rs</b> - Logic responsible for TVL and</li> </ul> | <p>Contract achieves this functionality.</p> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <p>bank mint price updating</p> <ul style="list-style-type: none"> <li>- <b>update_yield_info.rs</b> - Logic responsible for TVL updating when Oracle posts pending yield amount</li> <li>- <b>update_yield_amount.rs</b> - Push pending yield from external source to vault reserve</li> <li>- <b>team_withdraws_to_invest.rs</b> - Move funds from vault to external yield manager for investing</li> <li>- <b>team_deposits_from_invest.rs</b> - Return funds from external yield manager back to vault</li> <li>- <b>team_withdraws_fees.rs</b> - Withdraw accumulated fees from fee ATA to team</li> <li>- <b>vault_transfer.rs</b> - Transfer yielding tokens between vaults</li> </ul> |                                                     |
| <p><b>instructions/user/</b></p> <ul style="list-style-type: none"> <li>- <b>create_ua.rs</b> - Initialize UserGenState tracking account for a user</li> <li>- <b>mint_w_yielding.rs</b> - Deposit yielding tokens, mint bank tokens to user</li> <li>- <b>burn_for_yielding.rs</b> - Burn bank tokens, withdraw yielding tokens to user</li> </ul>                                                                                                                                                                                                                                                                                                                                           | <p><b>Contract achieves this functionality.</b></p> |
| <p><b>states/</b></p> <ul style="list-style-type: none"> <li>- <b>bank_state.rs</b> - Top-level state - vault registry, total TVL, issued supply, mint reference and managers</li> <li>- <b>vault/vault_gen_state.rs</b> - Vault config, accounting, fee settings and conversion logic</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                             | <p><b>Contract achieves this functionality.</b></p> |

|                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <ul style="list-style-type: none"> <li>- <b>vault/oracle_gen_state.rs</b> - Oracle registry, price and yield data arrays, consensus results and staleness parameters</li> <li>- <b>vault/user_gen_state.rs</b> - Per-user tracking - minted and burned amounts, TVL in and out, price bounds</li> <li>- <b>vault/team_gen_state.rs</b> - Team accounting - fees to withdraw and external yield tracking</li> </ul> |                                                     |
| <p><b>types/</b></p> <ul style="list-style-type: none"> <li>- <b>bool.rs</b> - Pod-safe boolean wrapper (PodBool) for zero_copy accounts</li> <li>- <b>enums.rs</b> - VaultType enum (YieldBearing, CollateralizedExternalYield, UnCollateralizedExternalYield), Action enum</li> </ul>                                                                                                                            | <p><b>Contract achieves this functionality.</b></p> |
| <p><b>utils/</b></p> <ul style="list-style-type: none"> <li>- <b>metadata.rs</b> - Create and update Metaplex metadata for bank mint token</li> </ul>                                                                                                                                                                                                                                                              | <p><b>Contract achieves this functionality.</b></p> |

## Code Quality

This audit scope involves the smart contracts of the Perena project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

## Audit Resources

We were given the Perena project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

## Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

## Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

| Significance | Description                                                                                                                                                                                           |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High         | High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community. |
| Medium       | Medium-level difficulties should be solved before deployment, but won't result in loss of funds.                                                                                                      |
| Low          | Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.                                                                                        |
| Gas          | Gas Optimisations, issues, and inefficiencies.                                                                                                                                                        |
| QA           | Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.                            |

## Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

| Significance        | Description                                                                                                                                                                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Resolved</b>     | The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.                                                                                   |
| <b>Acknowledged</b> | The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception. |
| <b>Unresolved</b>   | The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.                                                                                                                                                                                               |

# Audit Findings

## High

**[H-01] vault\_gen\_state** - The `convert_base_units` divides in both directions and returns the wrong scaling

### Description

In `states/vault/vault_gen_state.rs` the `convert_base_units` divides when `from_units < to_units`. This under-scales values that should be up-scaled.

### Vulnerability Details

The function computes `base_units_diff` and calls `checked_div` for both positive and negative cases. When converting from fewer decimals to more decimals, it should multiply by  $10^{(to - from)}$ .

Returning a divided value propagates into mint and burn amount calculations and can zero out or shrink amounts by orders of magnitude. This creates consistent mis-pricing and fee errors.

```
fn convert_base_units(&self, value: u128, from_units: u8, to_units: u8) -> u128 {  
    let base_units_diff: i64 = (from_units as i64) - (to_units as i64);  
  
    if base_units_diff > 0 {  
        value.checked_div((10u64).pow(base_units_diff as u32) as u128).unwrap_or(0)  
    } else if base_units_diff < 0 {  
        value.checked_div((10u64).pow(base_units_diff.abs() as u32) as  
u128).unwrap_or(0)  
    } else {  
        value  
    }  
}
```

```
}  
  
}
```

### Impact

Users may be over- or under-credited during mint and burn, depending on the mint and yield token decimals.

### Recommendation

We recommend changing `convert_base_units` to multiply when `to_units` is greater than `from_units`. Ensure every call site passes the correct `from_units` and `to_units`.

### Status

Resolved

**[H-02] vault\_gen\_state** - Decimals conversion is misused in `calc_bank_mint_to_yielding` and `calc_yielding_to_bank_mint`

## Description

Both conversion call sites invert the direction of units. They then call the broken `convert_base_units`, which magnifies the error.

## Vulnerability Details

In `states/vault/vault_gen_state.rs` `calc_bank_mint_to_yielding` function, the `yielding_to_send_to_user` variable is already in yielding token base units. The code calls `convert_base_units` with `from_units` set to `bank_state.mint.decimals` and `to_units` set to `oracle_state.config.yielding_mint_decimals`.

In `calc_yielding_to_bank_mint` the `corresponding_bank_mint_amount` variable is already in bank units, but the function again converts using `yielding_mint_decimals` to bank decimals. These inversions produce systematic mis-scaling even after fixing the helper.

```
let yielding_to_send_to_user = corresponding_yielding_amount

    .checked_sub(fee)

    .ok_or(ErrorCode::MathOverflow)?;

let converted_yielding_to_send_to_user = self.convert_base_units(
    yielding_to_send_to_user,
    bank_state.mint.decimals,
    oracle_state.config.yielding_mint_decimals
);

return Ok((
    converted_yielding_to_send_to_user

    .try_into()
```

```

        .map_err(|_| ErrorCode::ConversionOverflow)?,

        fee.try_into().map_err(|_| ErrorCode::ConversionOverflow)?,

        corresponding_yielding_amount.try_into().map_err(|_|
ErrorCode::ConversionOverflow)?,

        dollar_value_bank_mint.try_into().map_err(|_|
ErrorCode::ConversionOverflow)?,

    ));

```

## Impact

Final minted or withdrawn amounts are wrong which shifts value between users and the pool.

## Recommendation

We recommend removing the extra conversions where values are already in target units. If conversion is needed, pass `from_units` and `to_units` that reflect the actual source and destination of the computed value.

## Status

Resolved

## [H-03] metadata#handle - Wrong PDA signer seeds break metadata ops

### Description

The `utils/metadata.rs` builds `bank_signer_seeds` without `bank_index` while the BankState PDA uses `BANK_STATE_SEED` and `bank_index`. The PDA cannot sign the metadata CPI.

### Vulnerability Details

The `SetMetadata` handle prepares signer seeds as `BANK_STATE_SEED` and bump only. The BankState PDA seed derivation includes `BANK_STATE_SEED` and `bank_index` then bump.

The Metaplex Create and Update CPIs require a valid PDA signature from `bank_state`. The missing `bank_index` causes signature failure and blocks create or update of metadata. The `SetMetadata` account constraints elsewhere show the correct seeds.

```
pub fn handle(ctx: Context<SetMetadata>, name: String, symbol: String, uri: String) ->
Result<()> {
    require!(
        ctx.accounts.payer.key() == ctx.accounts.bank_state.load()?.config.bank_manager,
        ErrorCode::Unauthorized
    );

    let bank_state = &ctx.accounts.bank_state;

    let bank_signer_seeds: &&[u8]] = &&[BANK_STATE_SEED,
&[bank_state.load()?.bump]];

    // use data_is_empty to check if metadata account is initialized:
https://medium.com/@Arrivant\_/how-to-verify-nfts-in-an-anchor-program-a051299acde8

    if ctx.accounts.metadata_account.data_is_empty() {
```

### Impact

Mint metadata cannot be created or updated. Any downstream client assumptions about metadata will fail.

## Recommendation

We recommend using the `bank_state.signer_seeds` from `BankState`. If not available, include `bank_index` in the seed array along with `BANK_STATE_SEED` and bump.

## Status

Resolved



## Medium

### [M-01] oracle\_gen\_state - Oracle staleness is not enforced in price or amount updates and user flows

#### Description

The program sets `price_max_ts_gap` but does not reject stale oracle data in `update_price`, `update_yield_info`, `mint`, or `burn`.

#### Vulnerability Details

The `OracleGenState` holds `price_max_ts_gap` and last timestamps per oracle. `UpdateYieldingPrice` and `UpdateYieldingInfo` update values and confirm equality, but do not compare `Clock::get` to `oracle_last_ts` before using or promoting a price or pending amount.

The user flows read `oracle_state.result` without staleness guards. Stale data acceptance creates value transfer risk and wrong `bank_mint` pricing.

```
oracle_account.result.price_update_ts = Clock::get()?.unix_timestamp as u64;  
oracle_account.config.price_max_ts_gap = 24 * 60 * 60;  
oracle_account.config.price_max_gap_bps = 10; // 0.1% price change  
oracle_account.config.yielding_mint_decimals = ctx.accounts.yielding_mint.decimals;
```

#### Impact

Users can deposit or withdraw against stale prices and pending yield. The pool can suffer unexpected value shifts.

#### Recommendation

We recommend enforcing `now` minus `oracle_last_ts` and `now` minus `result.price_update_ts` to be below `price_max_ts_gap` in every path that consumes `oracle_state` values. Reject operations if the bound is violated.

## Status

Resolved

**[M-02] update\_yield\_amount** - The `update_yield_amount` decrements both `oracle_pending_yield` slots blindly

## Description

After pushing yield, the handler subtracts from `oracle_pending_yield[0]` and `[1]` regardless of which oracle updated.

## Vulnerability Details

In `instructions/admin/actions/update_yield_amount.rs` handle the code that reduces `oracle_state.result.yielding_pending_amount` by `final_yield_to_push`. It then subtracts the same amount from `oracle_pending_yield[0]` and `oracle_pending_yield[1]` without ensuring symmetry.

If one oracle had not set that amount the subtraction underflows and reverts with `MathOverflow`. This creates a liveness and usability problem during normal posting cadence.

```
transfer_checked(cpi_ctx, final_yield_to_push, ctx.accounts.yielding_mint.decimals)?;

                                vault_state.accounting.yielding_reserve_amount      =
vault_state.accounting.yielding_reserve_amount

                                .checked_add(final_yield_to_push)

                                .ok_or(ErrorCode::MathOverflow)?;

                                oracle_state.result.yielding_pending_amount      =
oracle_state.result.yielding_pending_amount

                                .checked_sub(final_yield_to_push)

                                .ok_or(ErrorCode::MathOverflow)?;
```

```
oracle_state.data.oracle_pending_yield[0] = oracle_state.data.oracle_pending_yield[0]  
  
    .checked_sub(final_yield_to_push)  
  
    .ok_or(ErrorCode::MathOverflow)?;  
  
oracle_state.data.oracle_pending_yield[1] = oracle_state.data.oracle_pending_yield[1]  
  
    .checked_sub(final_yield_to_push)  
  
    .ok_or(ErrorCode::MathOverflow)?;
```

## Impact

Pushing yield can revert when the two oracle slots are not equal and operations stall.

## Recommendation

We recommend decrementing only the slot corresponding to the last confirmed value or require both slots to be equal to pending before subtraction. Add explicit guards to prevent underflow.

## Status

Resolved

## [M-03] bank\_state - No invariant check between bank\_mint.supply and internal issued supply

### Description

The program never compares SPL mint supply to `bank_state.accounting.total_issued_supply` after mint or burn.

### Vulnerability Details

The `BankState.sanitize_bank_mint_backing` checks vault tvls and `issued_supply` arrays but does not read the actual SPL mint supply. The mint and burn handlers update internal counters based on assumed CPI success.

If a CPI used a wrong program or failed to change state the mismatch persists and future sanity checks pass. This opens the door to long-lived accounting drift.

```
pub fn sanitize_bank_mint_backing(&self) -> Result<()> {

    let mut total_tv1 = 0_u64;

    let mut total_issued_bank_mint = 0_i64;

    for i in 0..MAX_SIZE_VAULT_REGISTRY as usize {

                                                total_tv1 =
total_tv1.checked_add(self.vaults.tvls[i]).ok_or(ErrorCode::MathOverflow)?;

        total_issued_bank_mint = total_issued_bank_mint
            .checked_add(self.vaults.issued_supply[i])
            .ok_or(ErrorCode::MathOverflow)?;
    }

    if total_tv1 != self.accounting.yielding_tv1 {

        msg!("Total TVL: {}, Bank TVL: {}", total_tv1, self.accounting.yielding_tv1);

        return err!(ErrorCode::BankAccountingIssue);
    }
}
```

```
}

if (total_issued_bank_mint as u64) != self.accounting.total_issued_supply {

    msg!(

        "Total issued bank_mint: {}, Bank issued bank_mint: {}",

        total_issued_bank_mint,

        self.accounting.total_issued_supply

    );

    return err!(ErrorCode::BankAccountingIssue);

}
```

## Impact

Internal accounting can diverge from true supply, which hides an inconsistent state or prior exploitation.

## Recommendation

We recommend that after mint and burn, reload `bank_mint` and compare supply to `total_issued_supply`. Add a periodic or per-instruction invariant check that aborts when a mismatch is detected.

## Status

Resolved

## [M-04] bankineco - TVL and supply caps are stored but never enforced

### Description

The program records `yielding_tvl_cap` and `supply_bank_mint_cap` in `BankState` and `VaultGenState`. No instruction aborts when a cap would be exceeded.

### Vulnerability Details

The program records `yielding_tvl_cap` and `supply_bank_mint_cap` in `BankState` and `VaultGenState`. The caps are initialized to zero and later configurable through admin flows. The mint and oracle update paths do not read these fields before adjusting totals.

This leads to a false sense of safety where UIs show caps, but the program does not apply them. Enforcement must happen in the on-chain handlers to be reliable.

```
#[derive(Default, TypeLayout)]  
  
pub struct BankAccounting {  
    pub yielding_tvl: u64,  
    pub total_issued_supply: u64,  
    pub yielding_tvl_cap: u64,  
    pub supply_bank_mint_cap: u64,  
    pub _padding1: [u64; 12],  
}
```

### Impact

The pool can grow beyond intended limits. Risk controls relying on caps are ineffective.

### Recommendation

We recommend adding checks in mint, burn, and price or yield update handlers that compare the new totals to caps and abort on overflow.

### Status

Resolved

## [M-05] update\_yield\_info - Division by zero when reserve is zero in update\_yielding\_info

### Description

The `update_yielding_info.rs` divides by `vault_state.accounting.yielding_reserve_amount`. This can be zero at initialization or after full withdrawal.

### Vulnerability Details

The `UpdateYieldingInfo` handle computes `increase_vs_confirmed_pending_bps` and `increase_vs_pending_one_bps` by dividing deltas by `yielding_reserve_i64`. When the reserve is zero, the division fails with `MathOverflow`, and the instruction reverts.

This blocks legitimate updates during early or empty reserve states and can halt operations that rely on fresh pending amounts.

```
let old_pending_two_i64 = oracle_state.data.oracle_pending_yield

    [index_relevant_other_oracle] as i64;

let yielding_reserve_i64 = vault_state.accounting.yielding_reserve_amount as i64;

let max_gap_bps_i64 = oracle_state.config.price_max_gap_bps as i64;

let pending_yield_i64 = pending_yield as i64;

let increase_vs_confirmed_pending_bps = pending_yield_i64

    .checked_sub(old_confirmed_pending_amount_i64)

    .and_then(|x| x.checked_mul(BPS_MULTIPLIER_I64))

    .and_then(|x| x.checked_div(yielding_reserve_i64))

    .ok_or(ErrorCode::MathOverflow)?;

let increase_vs_pending_one_bps = pending_yield_i64

    .checked_sub(old_pending_one_i64)

    .and_then(|x| x.checked_mul(BPS_MULTIPLIER_I64))

    .and_then(|x| x.checked_div(yielding_reserve_i64))
```

```
.ok_or(ErrorCode::MathOverflow)?;
```

### Impact

Oracles cannot post pending amounts when the reserve is zero.

### Recommendation

We recommend short-circuiting when the reserve is zero and either accept only zero updates or defer comparison. Resume normal checks once the reserve is nonzero.

### Status

Resolved

## [M-06] update\_yield\_info - The update\_yielding\_amount lacks role binding and can be called by anyone

### Description

The `UpdateYieldingAmount` uses a generic signer and does not require an external yield manager or admin role.

### Vulnerability Details

The accounts for `UpdateYieldingAmount` define signer as `Signer` without any constraint that ties it to `team_state.details.external_yield_manager` or a bank manager. The handler then transfers tokens from `signer_ata` to the vault and mutates oracle pending amounts and reserve.

An attacker can grief the system by pushing arbitrary amounts and disturbing the expected timing.

```
pub struct UpdateYieldingAmount<'info> {  
  
    #[account(mut)]  
  
    pub signer: Signer<'info>,  
}
```

### Impact

Untrusted users can influence pending yield queues and reserve accounting.

### Recommendation

We recommend constraining the signer to equal `team_state.details.external_yield_manager` or `bank_state.config.bank_manager`. Enforce the same identity checks used in team deposit and withdrawal flows.

### Status

Acknowledged

# Low

## [L-01] `change_fees#handle` - Fees can be configured to 100 percent

### Description

The `ChangeFees` handle validates that each provided bps is less than or equal to 10000. The `vault_state.config` values are updated without additional guardrails. A misconfiguration or malicious management can set `minting_fee_bps`, `burning_fee_bps`, `performance_fee_bps`, or `managing_fee_bps` to 100 percent. This effectively blocks fair use or traps funds economically.

```
pub fn handle(
    ctx: Context<ChangeFees>,
    performance_fee_bps: u16,
    managing_fee_bps: u16,
    minting_fee_bps: u16,
    burning_fee_bps: u16
) -> Result<()> {
    let vault_state = &mut ctx.accounts.vault_state.load_mut()?;

    if
        performance_fee_bps > 10000 ||
        managing_fee_bps > 10000 ||
        minting_fee_bps > 10000 ||
        burning_fee_bps > 10000
    {
        return Err(ErrorCode::InvalidArgument.into());
    }
}
```

```
vault_state.config.performance_fee_bps = performance_fee_bps;  
  
vault_state.config.managing_fee_bps = managing_fee_bps;  
  
vault_state.config.minting_fee_bps = minting_fee_bps;  
  
vault_state.config.burning_fee_bps = burning_fee_bps;
```

### **Recommendation**

We recommend introducing per-fee upper bounds that are materially lower. Require multi-party approval or a time-lock for fee increases that pass a threshold.

### **Status**

Resolved

## **[L-02] create\_bank** - No `mint_authority` and `freeze_authority` sanity checks at bank creation

### **Description**

The `create_bank.rs` records the mint but does not verify `mint_authority` or `freeze_authority`.

The program later assumes that the `BankState` PDA can mint. If the authority is not set, the mint step fails cleanly and reverts. A third-party freeze authority is a governance and UX risk rather than an immediate exploit path because freezes block usage rather than allow theft. Explicit checks during setup would catch misconfiguration early. This is a configuration hardening measure.

### **Recommendation**

We recommend that at creation, require `mint_authority` to equal `bank_state` PDA and `freeze_authority` to be `None` or equal to a controlled PDA. Document that these conditions must remain invariant.

### **Status**

Resolved

## [L-03] oracle\_gen\_state - Oracle activation and veto flags are not enforced in authorization

### Description

In `states/vault/oracle_gen_state.rs` the helper returns `Ok` when the signer is present in `oracle_registry`. The function does not read `oracle_activated` or `veto` bits. As a result, a deactivated or vetoed oracle can still post updates through `update_yielding_info` or `update_yielding_price`. This defeats governance controls intended by the state's layout.

```
pub fn check_if_registered_oracle(&self, signer: &Pubkey) -> Result<()> {  
    let signer_found = self.data.oracle_registry  
        .iter()  
        .any(|&oracle_pubkey| &oracle_pubkey == signer);  
    if !signer_found {  
        return Err(ErrorCode::Unauthorized.into());  
    }  
    Ok(())  
}
```

### Recommendation

We recommend enforcing `oracle_activated` equals one and `oracle_veto` equals zero for the matched index during authorization checks.

### Status

Resolved

## [L-04] `update_price` - Equality-only price confirmation without freshness gate

### Description

The `admin/actions/update_price.rs` promotes a price only when the basis-point difference is exactly zero and does not enforce `price_max_ts_gap` at promotion time.

The equality requirement is brittle and may stall updates on tiny rounding differences. The lack of a timestamp freshness check when moving values from `oracle_prices` into `result.yielding_token_price` increases the chance of accepting stale agreement.

This does not enable theft but creates liveness and valuation quality issues that can hurt users indirectly. The code already stores timestamps and a gap parameter, so the fix should not be problematic.

### Recommendation

We recommend allowing confirmation within `price_max_gap_bps` and enforcing that both sources are fresh under `price_max_ts_gap` before promoting.

### Status

Acknowledged

## [L-05] team\_withdraws\_fees - Fee ATA authority is a program PDA rather than the human team signer

### Description

In `instructions/admin/actions/team_withdraws_fees.rs` the transfer of fees uses `team_state` as authority, and `fee_team_ata` is created with `team_state` as the owner. If `team_state` is rotated or the PDA seeds change, the underlying funds become inaccessible.

This is an operational custody risk that can lock fees and harm operations.

```
#[account(mut,
    constraint = team_state.load()?.details.fee_gatherer == team.key(),
    seeds = [TEAM_ACCOUNT_SEED, vault_state.key().as_ref()],
    bump = team_state.load()?.bump
)]

pub team_state: AccountLoader<'info, TeamGenState>,

#[account(associated_token::mint = yielding_mint, associated_token::authority =
team_state)]

pub fee_team_ata: Box<InterfaceAccount<'info, TokenAccount>>,
```

### Recommendation

We recommend setting `fee_team_ata` authority to a human key or a multisig, and keep `team_state` only as a control signer for instruction authorization.

### Status

Acknowledged

## [L-06] create\_oracle\_state - Zero initial price is accepted and causes division by zero later

### Description

The `CreateOracleState` handle writes `contract_init_price` to both oracle slots and to `result.yielding_token_price`. `UpdateYieldingPrice` later divides by `old_two_price_i64` when computing `two_oracles_prices_diff` and other ratios.

If the initial price is zero, these divisions fail with `MathOverflow`. This misconfiguration can brick the price pipeline until a state migration.

```
oracle_account.data.oracle_registry[0] = oracle_one;

    oracle_account.data.oracle_prices[0] = contract_init_price;

    oracle_account.data.oracle_activated[0] = 1;

    oracle_account.data.oracle_last_ts[0] = current_ts;


    oracle_account.data.oracle_registry[1] = oracle_two;

    oracle_account.data.oracle_prices[1] = contract_init_price;

    oracle_account.data.oracle_activated[1] = 1;

    oracle_account.data.oracle_last_ts[1] = current_ts;


    oracle_account.result.yielding_token_price = contract_init_price;
```

### Recommendation

We recommend requiring `contract_init_price` to be greater than zero during Oracle state creation. Add guardrails to reject zero or nonsensical prices in updates as well.

### Status

Resolved

**[L-07]**      **create\_gen\_vault**      -      ATA      creation      omits  
associated\_token::token\_program for Token-2022 mints

## Description

In `create_gen_vault` and `create_team_state`, the `yielding_vault_ata` and `fee_team_ata` are created without specifying the token program. Other instructions pass the token program explicitly, which shows the correct pattern.

Without this field, the wrong ATA program can be selected, and later CPIs will fail for `Token-2022`. This creates inconsistent behavior across instructions.

## Recommendation

We recommend always passing `associated_token::token_program` equal to the `TokenInterface` program param when creating ATAs. Standardize this across all instructions.

## Status

Resolved

## [L-08] name\_bank\_manager - Missing mut annotation while calling load\_mut in NameBankManager

### Description

The `NameBankManager` handles writes `bank_state.config.bank_manager`. The `Accounts` struct defines `bank_state` with PDA seeds but omits the `mut` attribute. Anchor expects writable accounts to be flagged for runtime access.

Without `mut`, the instruction can fail at runtime or rely on clients to mark the account writable, which is fragile.

```
pub fn handle(ctx: Context<NameBankManager>, bank_manager: Pubkey) -> Result<()> {
    let bank_state = &mut ctx.accounts.bank_state.load_mut()?;

    require!(ctx.accounts.creator.key() == bank_state.config.creator,
    ErrorCode::Unauthorized);

    bank_state.config.bank_manager = bank_manager;

    Ok(())
}

#[derive(Accounts)]

pub struct NameBankManager<'info> {
    #[account(
        seeds = [BANK_STATE_SEED, &bank_state.load()?.bank_index.to_le_bytes()],
        bump = bank_state.load()?.bump
    )]

    pub bank_state: AccountLoader<'info, BankState>,

    #[account(mut,
        constraint = creator.key() == bank_state.load()?.config.creator,
    )]
```

## Recommendation

We recommend adding `#[account(mut, ...)]` to `bank_state` in `NameBankManager`. Keep `mut` consistent anywhere `load_mut` is used.

## Status

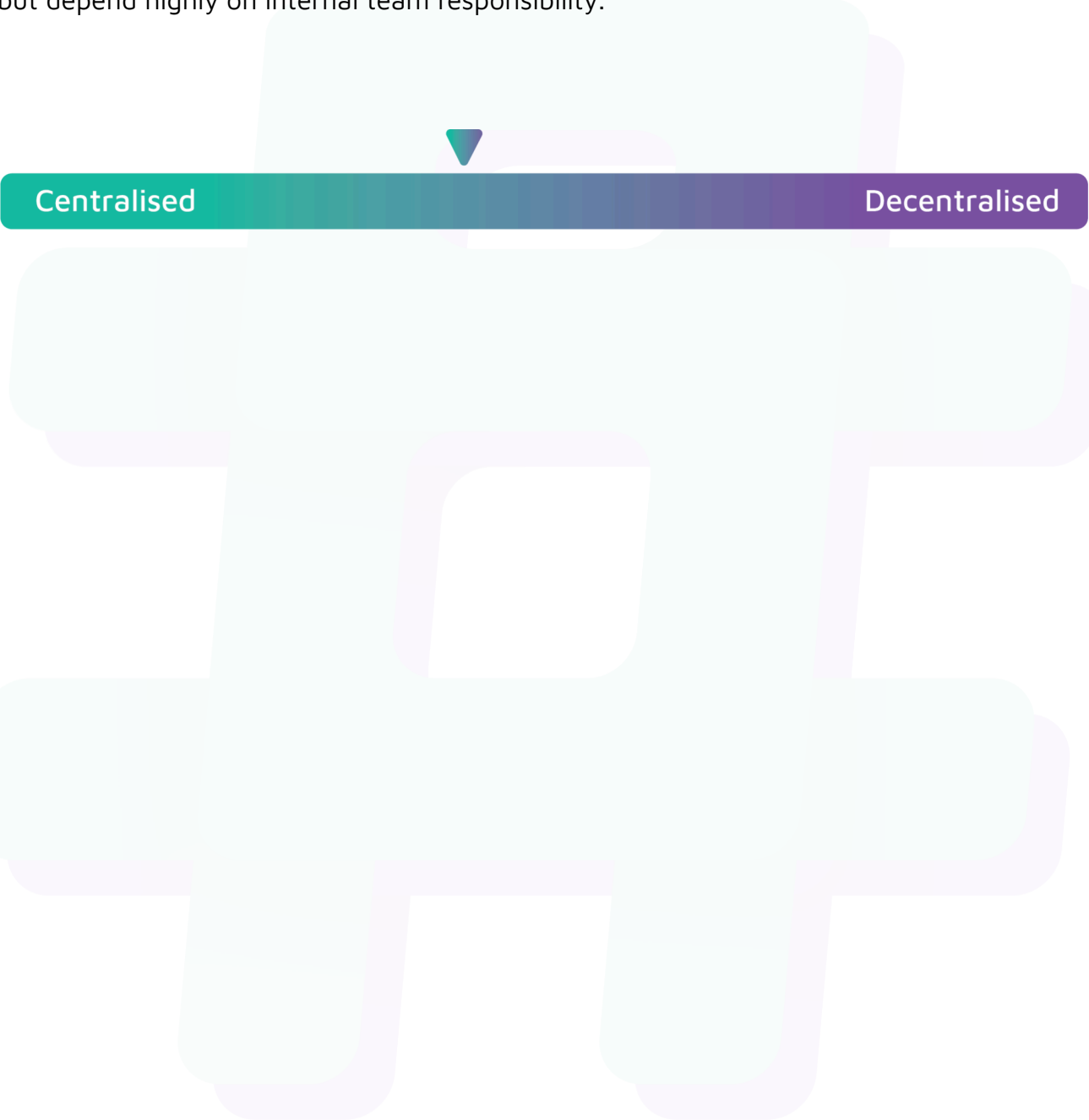
Resolved



## Centralisation

The Perena project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

## Conclusion

After Hashlock's analysis, the Perena project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

## Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

### **Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

### **Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

## About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** [hashlock.com.au](https://hashlock.com.au)

**Contact:** [info@hashlock.com.au](mailto:info@hashlock.com.au)



**#hashlock.**